

Sweep Line Algorithms

Design and Analysis of Algorithms

Brian C. Dean

**School of Computing
Clemson University**

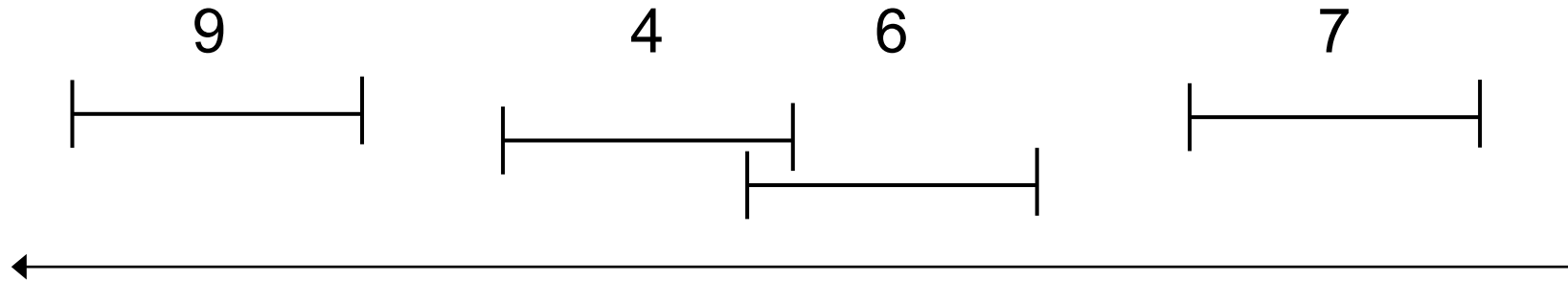


Sort First, Ask Questions Later...

- Many problems become easier after first sorting the input as a preprocessing step.
- Sort and scan or sweep line algorithms are particularly common in computational geometry, and also a good example of data structure usage in algorithmic problem solving.

Warm-Up

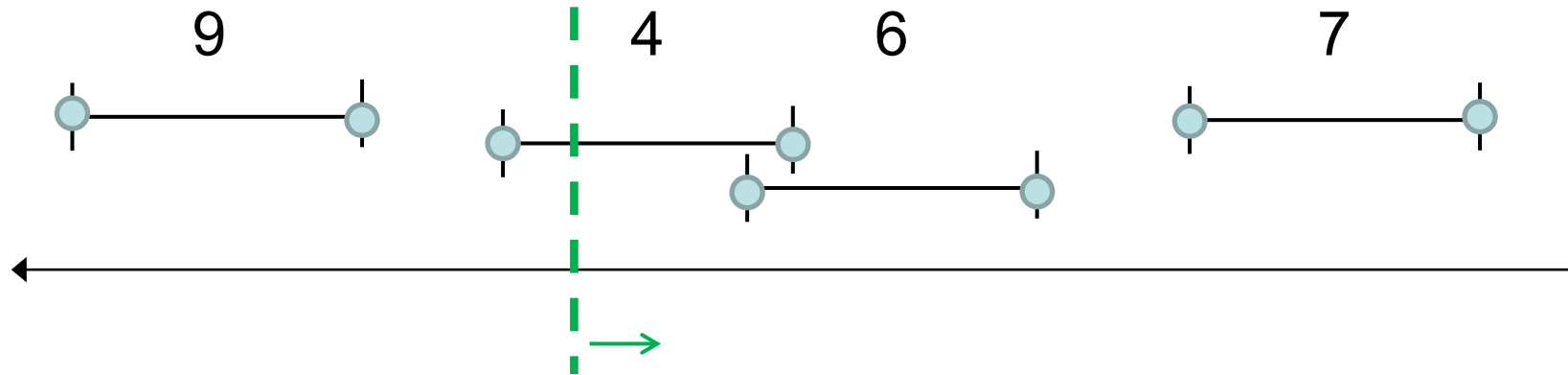
- You are told N intervals on the number line, each with an associated value.



- Find a point of maximum overlap (i.e., maximizing the sum of interval values overlapping at that point).

A “Sweep Line” Approach

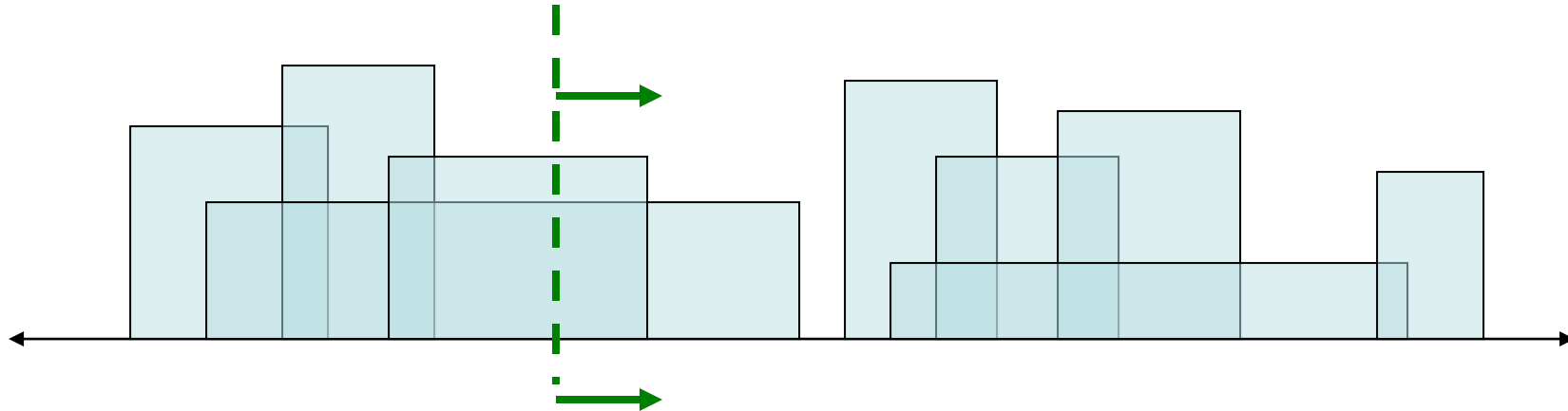
- You are told N intervals on the number line, each with an associated value.



- Find a point of maximum overlap (i.e., maximizing the sum of interval values overlapping at that point).

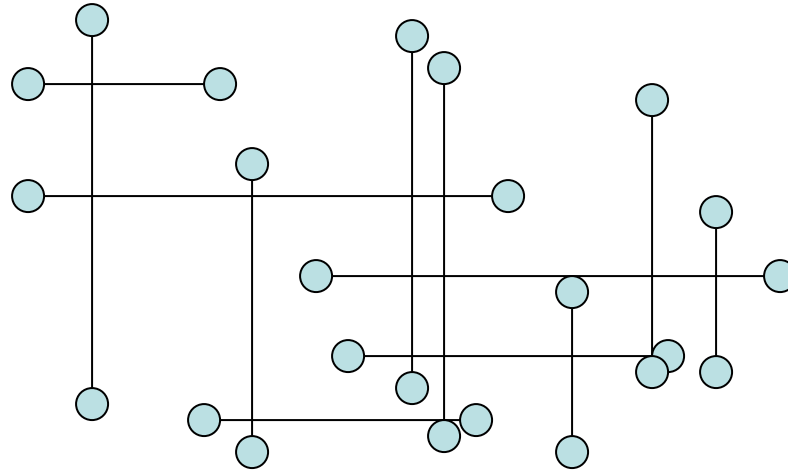
Example: The Skyline Problem (also from Module 3)

- Give N rectangular buildings sharing a common base, find the area of the “skyline” they form.



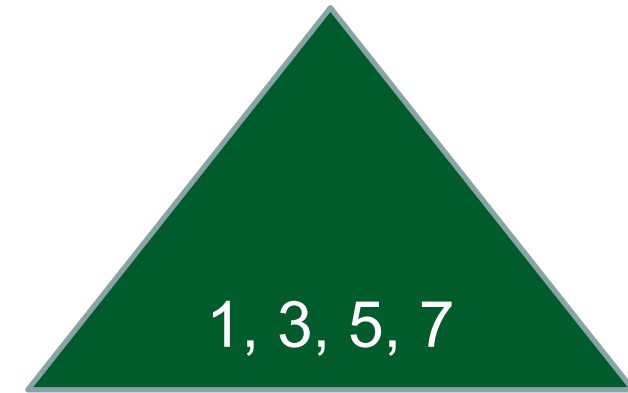
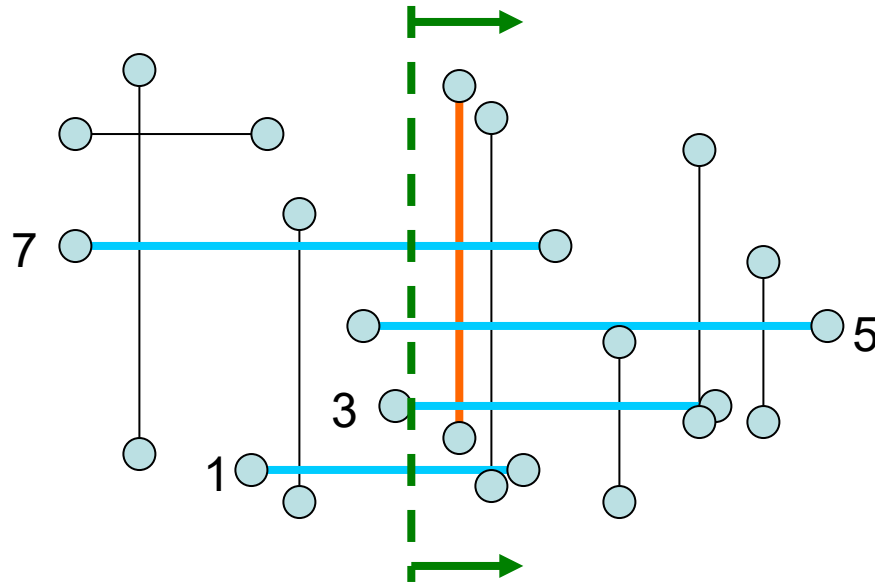
- Sweep line + binary heap (or balanced BST).
- N inserts, N deletes, $2N$ find-max: $O(N \log N)$

Counting Intersection Points Among Axially-Aligned Segments



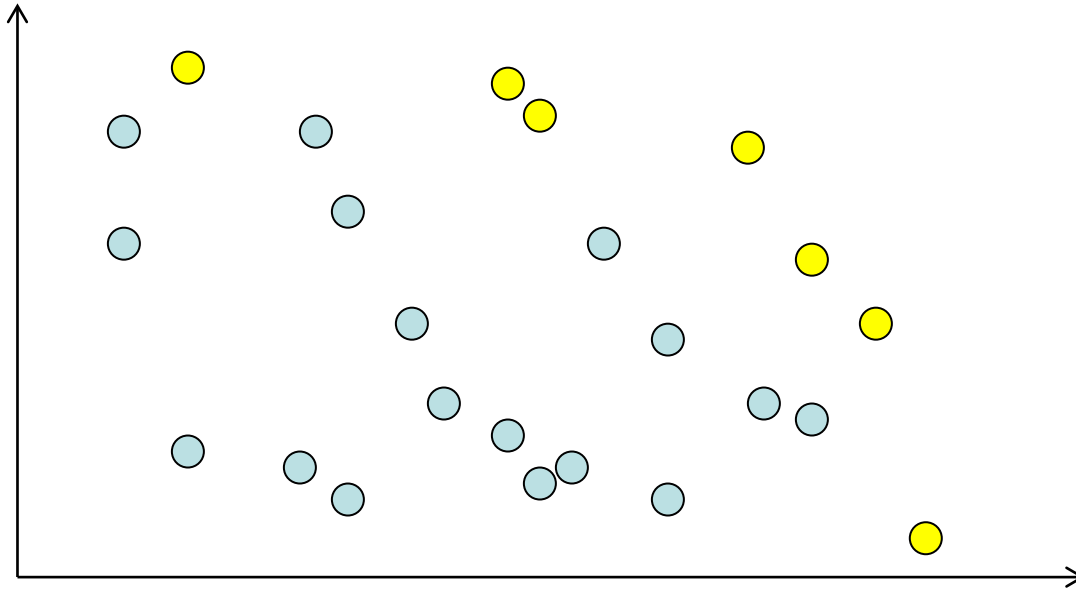
Important note: We are doomed to take $\Theta(N^2)$ time if we count intersections one by one.

Counting Intersection Points Among Axially-Aligned Segments

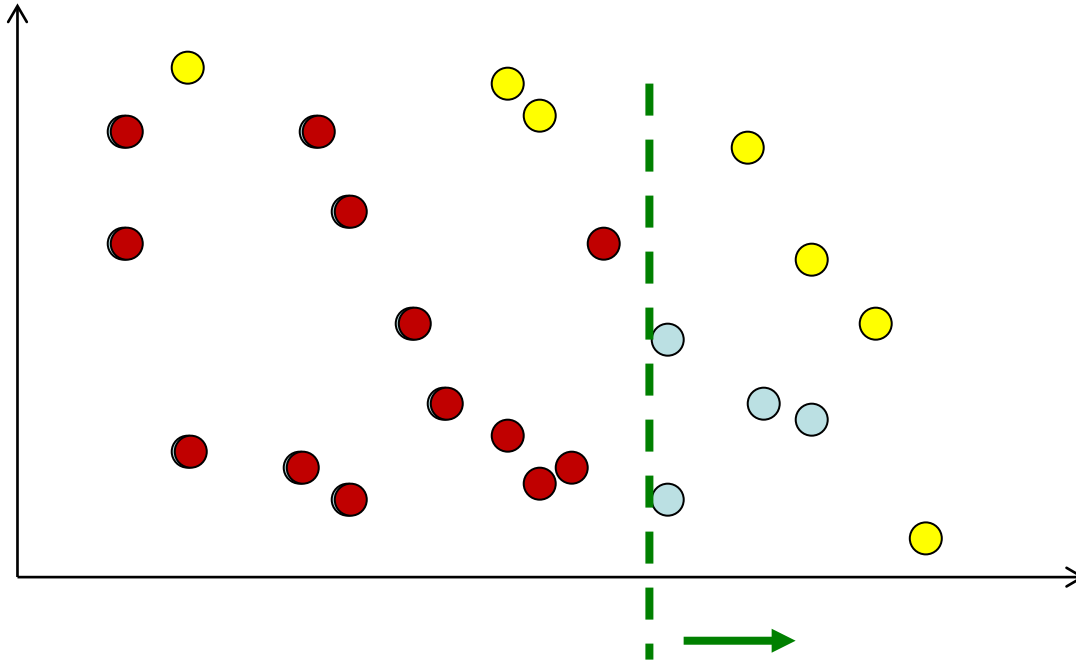


- Sweep line on x + balanced BST of “active” horizontal segments, keyed on y .
- $\leq 2N$ inserts, $\leq 2N$ deletes, $\leq N$ range counting queries: $O(N \log N)$
(# elements in range $[x, y] \approx \text{rank}(y) - \text{rank}(x)$)

Identifying Non-Dominated Points (Module 2)

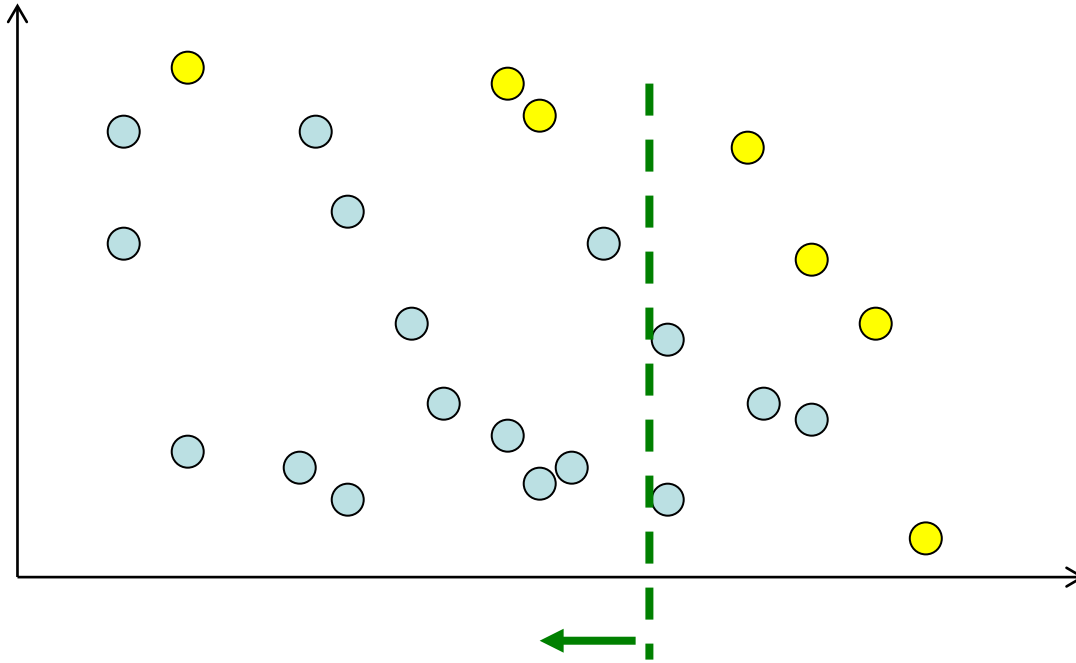


Identifying Non-Dominated Points



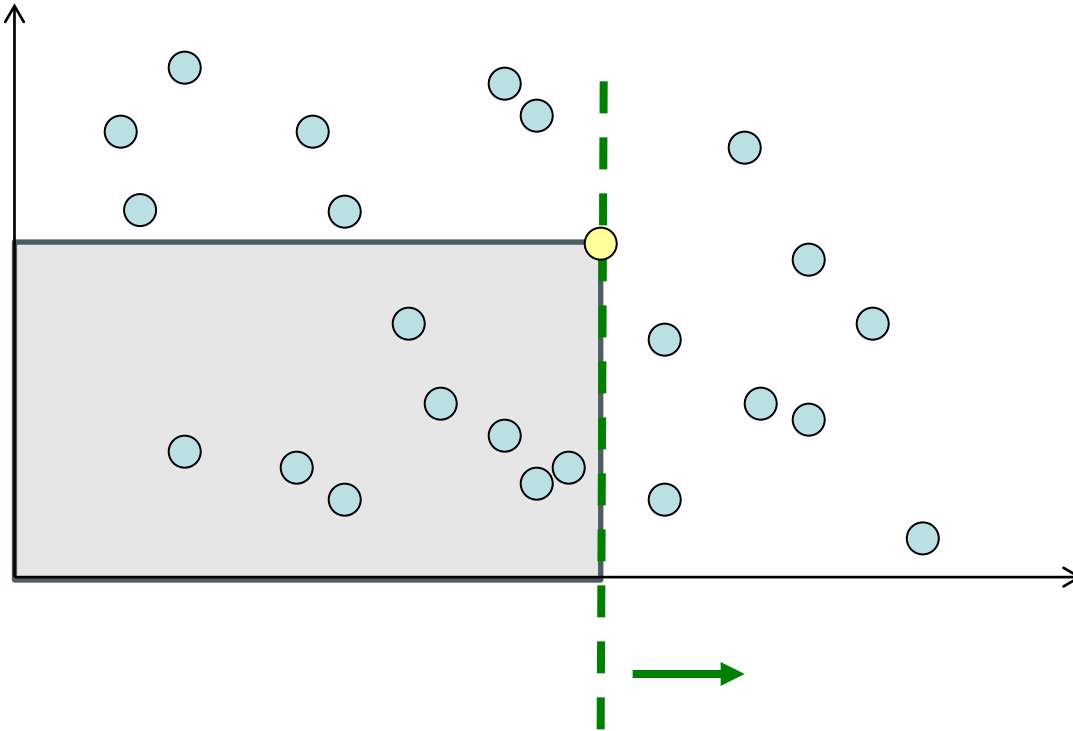
- Forward sweep line on x + heap or BST on y.
- N inserts, $\leq N$ deletes: $O(N \log N)$

Identifying Non-Dominated Points



- Reverse sweep line on x + running max on y.
- $O(N \log N)$

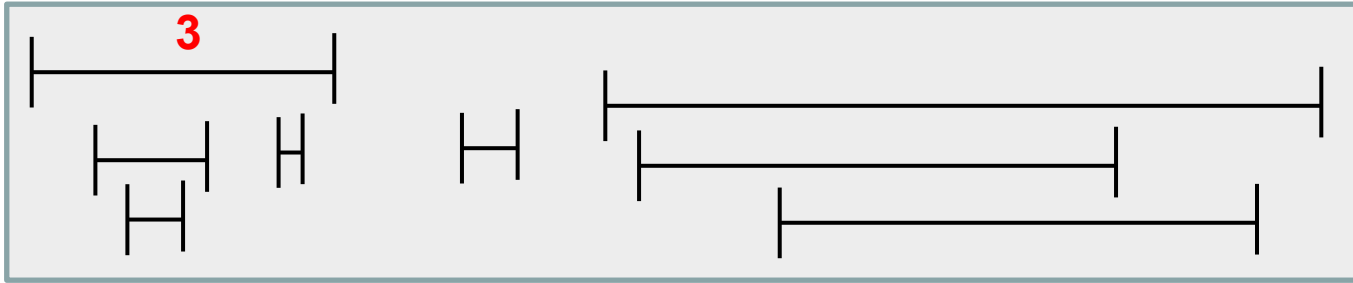
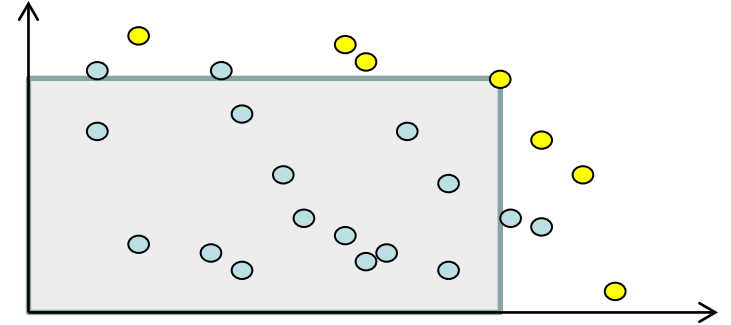
New: Finding the “Domination Count” for Every Point



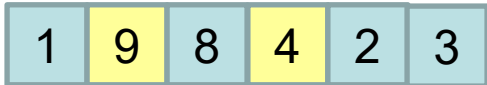
- Forward scan on x + BST on y with calls to rank
- $O(N \log N)$

Four Interesting Problems

1. Compute the “domination count” for each point.

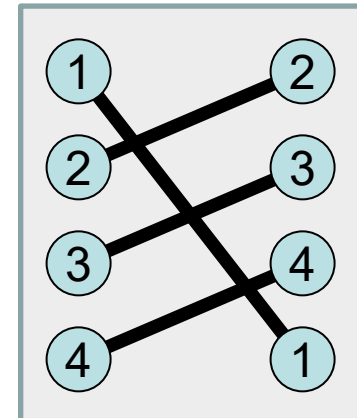


2. Given N intervals, compute for each the number of intervals contained within it.

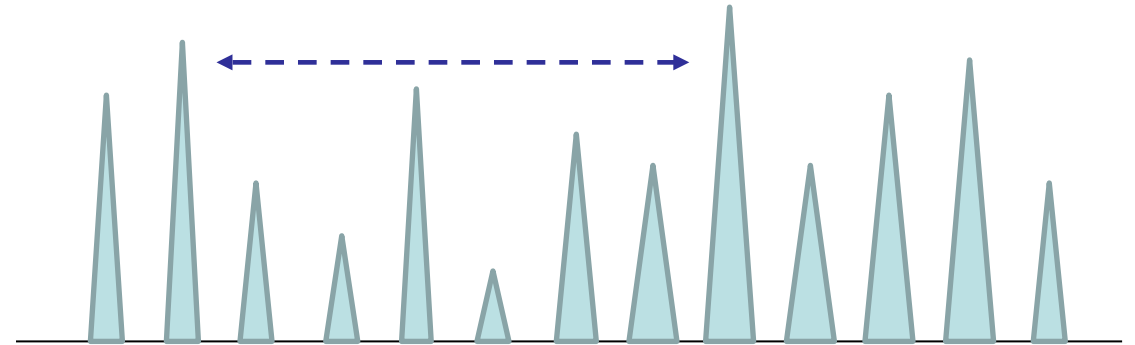
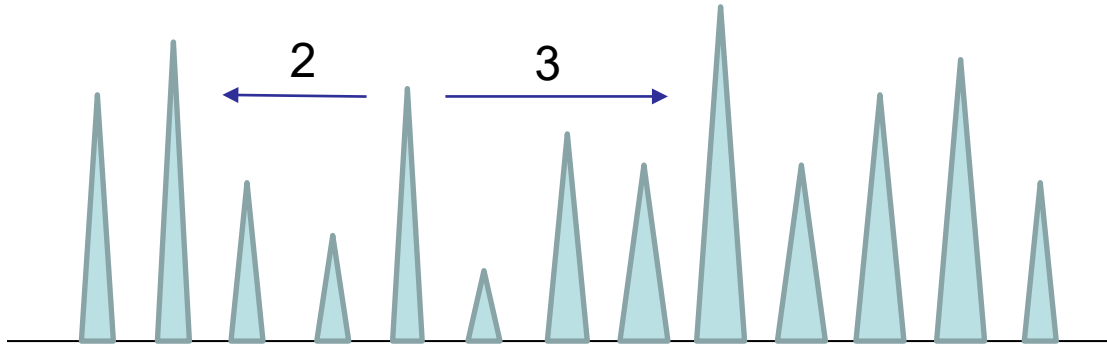


3. Count “inversions” in an N-element array

4. Count the pairs of crossing segments here:

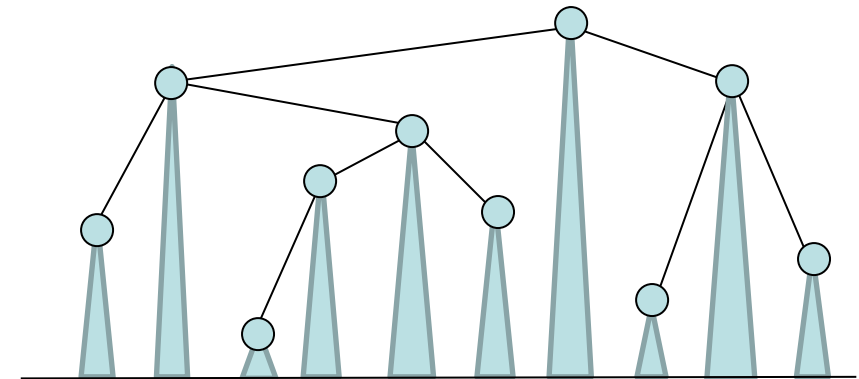


Example: Domination Radius / Longest Line of Sight

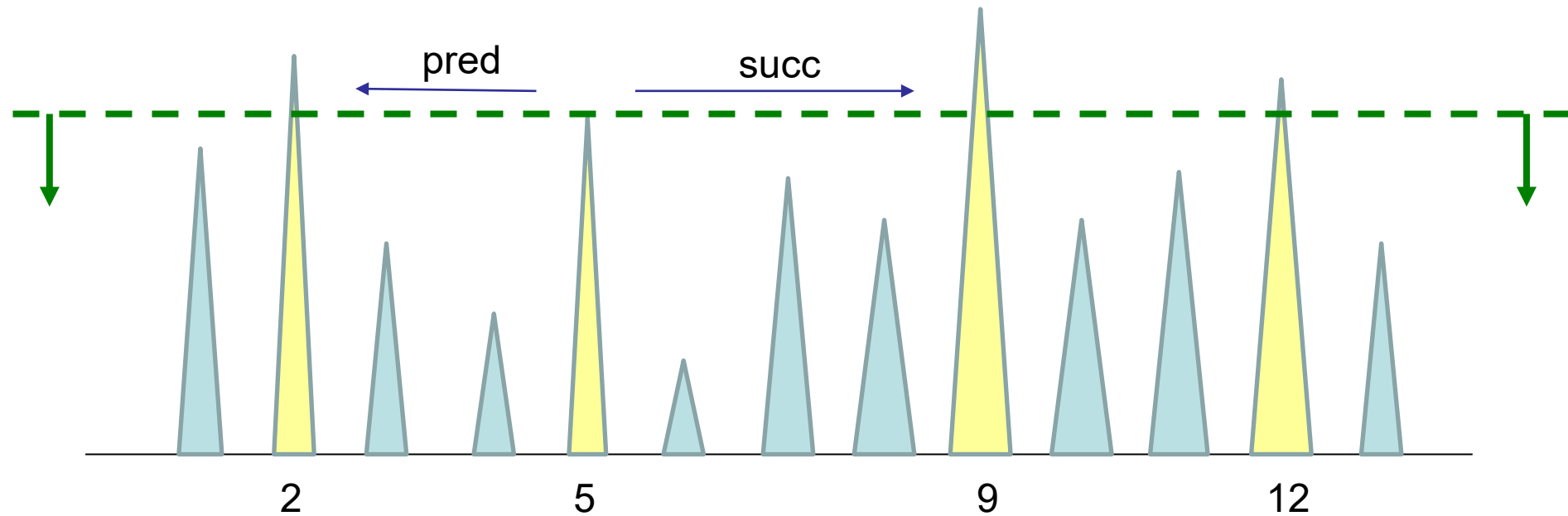


We've looked at this problem quite a bit already...

- Module 1: $O(N)$ in coding discussion (using what could be interpreted as a form of sweep line algorithm)
- Module 2: $O(N \log N)$ average-case time for simplistic “scan left, then scan right” method; connections with randomized quicksort (and alluding to connections with binary search trees)
- Module 3: $O(N \log N)$ for longest line of sight via divide and conquer



Example: Domination Radius / Longest Line of Sight



$O(N \log N)$: Sweep downward, storing x positions in balanced BST. Left and right lines of sight given by calls to `pred` and `succ`.

Find Closest Pair of Points in 2D

